9、串口驱动

全志V3S嵌入式驱动开发（串口驱动）

原创

嵌入式-老费Open: 7f185b501ef38a059b10d1b79d7cfdb9_MD5.png

已于 2023-08-29 14:48:51 修改

Open: 8a923f94c34ac151897c6be747452d7c_MD5.png

阅读量1k Open: 196806fd3bdd4f04e1a92a7697cf0354_MD5.png

收藏 1

Open: 95eb4f436541fdf1b2e1af49844ad0c3_MD5.png

点赞数

CC 4.0 BY-SA版权

分类专栏： 基于全志soc的linux嵌入式开发 文章标签： 驱动开发

Open: 135c7190ea3ccdc82fa1f0c0a41d08f0_MD5.png

基于全志soc的linux嵌入式开发专栏收录该内容](https://blog.csdn.net/feixiaoxing/category_12270221.html "基于全志soc的linux嵌入式开发")

73 篇文章¥49.90¥99.00

已订阅8折续费

[

Open: d9b05f322fadabcd81dc6c5654189e59_MD5.png

您已是超级会员，正在免费阅读会员专享内容

查看更多超级会员权益Open: ddf4d655e9aa3806864ef10c7c4a060b_MD5.png

](https://www.csdn.net/vip?utm_source=bkzl_cjhy_ckqy)

Open: 1e4395309f4514fed3e1f5e89b2a25e6_MD5.png

摘要 本文介绍了全志V3S芯片中UART0和UART2的驱动开发过程，包括电路图分析、设备树配置、驱动文件详解及测试方法。在电路分析中，强调了UART0和UART2的连接方式。在设备树配置中，指出必须为UART2添加配置。在驱动文件部分，指出了驱动文件的位置和内容。最后，提供了测试代码和测试步骤，以实现UART2的数据回显功能，强调了掌握串口编程在嵌入式开发中的重要性。

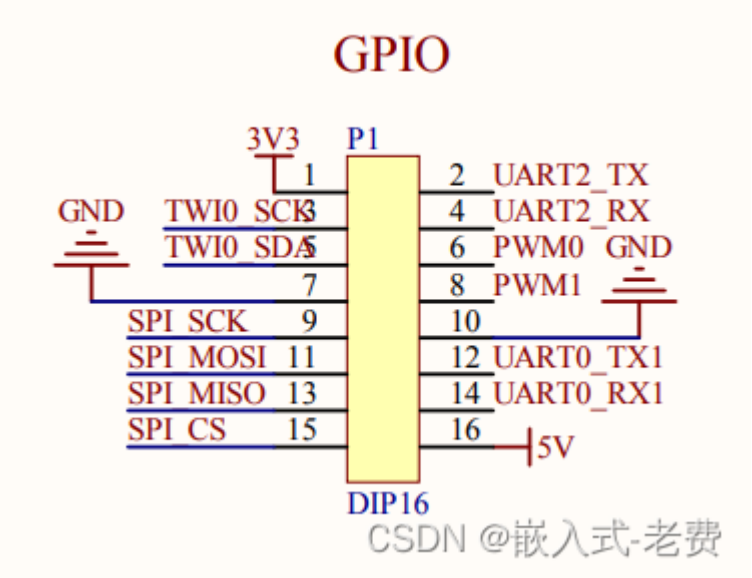
摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

【 声明：版权所有，欢迎转载，请勿用于商业用途。 联系信箱：feixiaoxing@163.com】

全志V3S支持三个串口，但是因为其中UART1的pin和其他功能是复用的，所以这个时候一般只用UART0和UART2。当然我们在linux开发必须要进行调试，这就涉及到shell命令，因此就会占用一个串口UART0，所以如果有外接设备需要对接的话，一般用UART2。

1、电路图分析

Open: 891d40bd5650e24e02ef5abdaa1987d1_MD5.png



如上述电路图所示，分别有两组UART，一个是UART0，一个是UART2。连线的时候，通常10、12、14一组，2、4、7算成一组，这样就基本没有问题了。GND一定要连接上。

2、添加设备树配置

设备树当中，默认UART0是配置好的，不然在启动启动的时候，也看不到那么多的日志信息。只是如果需要使用UART2的话，那么需要分别在 sun8i-v3s.dtsi添加，

```
uart1_pins_a: uart1@0 {
    pins = "PE21", "PE22";
    function = "uart1";
    bias-pull-up;
};

uart2_pins_a: uart2@0 {
    pins = "PB0", "PB1";
    function = "uart2";
    bias-pull-up;
};
```

在 sun8i-v3s-licheepi-zero.dts添加，

```
    &uart1 {
        pinctrl-0 = <&uart1_pins_a>;
        pinctrl-names = "default";
        status = "okay";
    };

    &uart2 {
        pinctrl-0 = <&uart2_pins_a>;
        pinctrl-names = "default";
        status = "okay";
    };
```

有的同学也许会说，既然只使用到了UART2，那么UART1相关的配置不添加，行不行。我的实验结果告诉大家，不行。

配置文件修改后，大家把配置重新编译一下，就可以生成新的dtb文件，拷贝到sd卡即可。代码部分本身zImage里面就有，这里就不需要重新编译了。

3、驱动文件

细心一点的同学可能会在 sun8i-v3s.dtsi看到这些内容，这就是将来UART驱动所要读取的信息，包括了兼容设备、寄存器、中断、时钟、复位、状态等数据。

```
uart0: serial@01c28000 {
        compatible = "snps,dw-apb-uart";
        reg = <0x01c28000 0x400>;
        interrupts = <GIC_SPI 0 IRQ_TYPE_LEVEL_HIGH>;
        reg-shift = <2>;
        reg-io-width = <4>;
        clocks = <&ccu CLK_BUS_UART0>;
        resets = <&ccu RST_BUS_UART0>;
        status = "disabled";
};

uart1: serial@01c28400 {
        compatible = "snps,dw-apb-uart";
        reg = <0x01c28400 0x400>;
        interrupts = <GIC_SPI 1 IRQ_TYPE_LEVEL_HIGH>;
        reg-shift = <2>;
        reg-io-width = <4>;
        clocks = <&ccu CLK_BUS_UART1>;
        resets = <&ccu RST_BUS_UART1>;
        status = "disabled";
};

uart2: serial@01c28800 {
        compatible = "snps,dw-apb-uart";
        reg = <0x01c28800 0x400>;
        interrupts = <GIC_SPI 2 IRQ_TYPE_LEVEL_HIGH>;
        reg-shift = <2>;
        reg-io-width = <4>;
        clocks = <&ccu CLK_BUS_UART2>;
        resets = <&ccu RST_BUS_UART2>;
        status = "disabled";
};
```

根据前面的经验，我们可以通过全局搜索dw-apb-uart找到对应的驱动文件。当然，也可以到内核drivers/tty目录下，借助于find . -name "*.o"的方法，看看哪一个驱

动最像我们要找的那个文件。简单查看下，发现有一个drivers/tty/serial/8250/8250_dw.c最像，因为它也包含了一个dw字母。

```
static const struct of_device_id dw8250_of_match[] = {
    { .compatible = "snps,dw-apb-uart" },
    { .compatible = "cavium,octeon-3860-uart" },
    { /* Sentinel */ }
};
MODULE_DEVICE_TABLE(of, dw8250_of_match);

static const struct acpi_device_id dw8250_acpi_match[] = {
    { "INT33C4", 0 },
    { "INT33C5", 0 },
    { "INT3434", 0 },
    { "INT3435", 0 },
    { "80860F0A", 0 },
    { "8086228A", 0 },
    { "APMC0D08", 0},
    { "AMD0020", 0 },
    { "AMDI0020", 0 },
    { "HISI0031", 0 },
    { },
};
MODULE_DEVICE_TABLE(acpi, dw8250_acpi_match);

static struct platform_driver dw8250_platform_driver = {
    .driver = {
        .name       = "dw-apb-uart",
        .pm     = &dw8250_pm_ops,
        .of_match_table = dw8250_of_match,
        .acpi_match_table = ACPI_PTR(dw8250_acpi_match),
    },
    .probe          = dw8250_probe,
    .remove         = dw8250_remove,
};

module_platform_driver(dw8250_platform_driver);

MODULE_AUTHOR("Jamie Iles");
MODULE_LICENSE("GPL");
```

```
MODULE_DESCRIPTION("Synopsys DesignWare 8250 serial port driver");
MODULE_ALIAS("platform:dw-apb-uart");
```

看了一下驱动末尾的几段代码，应该就是这个文件。有兴趣的同学可以通过调试和打印，加深一下印象。

4、准备测试代码、开始测试

有了编译好的dtb文件，重新烧录后，下面就可以开始测试了。 **我们的测试方法就是，准备一个ttyS2的读写程序，此外再准备两个usb2ttl，一个用作shell输入，一个用作效果验证。** 网上关于linux串口编程的代码很多，可以找一个适合我们的查看下即可，

```
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <termios.h>
#include <errno.h>

int set_opt(int,int,int,char,int);
void main()
{
    int fd,nByte,flag=1;
    char *uart2 = "/dev/ttyS2";
    char buffer[512];
    char *uart_out = "Please input,waiting....\r\n";
    char *uart_demo = "Linux uart demo\r\n";
    char *line = "\r\n";
    memset(buffer, 0, sizeof(buffer));

    if((fd = open(uart2, O_RDWR|O_NONBLOCK))<0)//非阻塞读方式
        printf("open %s is failed",uart2);
    else{
        set_opt(fd, 115200, 8, 'N', 1);
        write(fd,uart_demo, strlen(uart_demo));
        write(fd,uart_out, strlen(uart_out));

        while(1){
            while((nByte = read(fd, buffer, 512))>0){
```

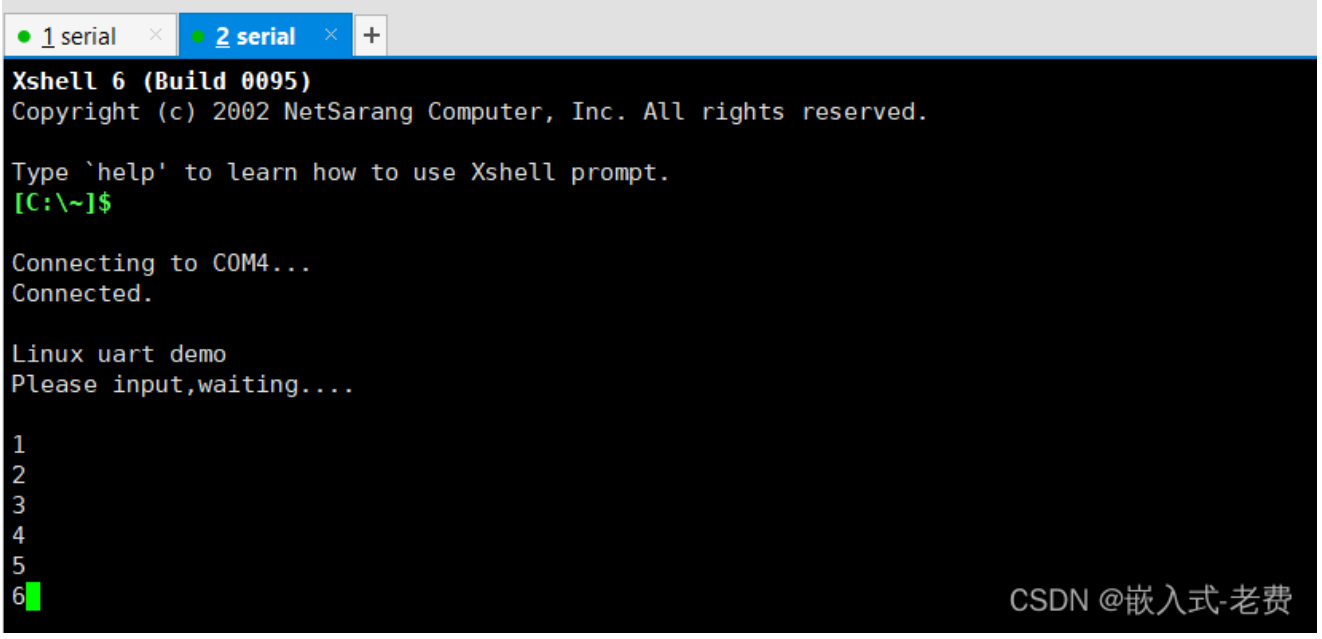
```
                buffer[nByte+1] = '\0';
                write(fd, line, strlen(line));
                write(fd,buffer,strlen(buffer));
                memset(buffer, 0, strlen(buffer));
                nByte = 0;
            }
        }
    }
}

int set_opt(int fd,int nSpeed, int nBits, char nEvent, int nStop)
{
    struct termios newtio,oldtio;
    if  ( tcgetattr( fd,&oldtio)  !=  0) {
        perror("SetupSerial 1");
        return -1;
    }
    bzero( &newtio, sizeof( newtio ) );
    newtio.c_cflag  |=  CLOCAL | CREAD;
    newtio.c_cflag &= ~CSIZE;

    switch( nBits )
    {
        case 7:
            newtio.c_cflag |= CS7;
            break;
        case 8:
            newtio.c_cflag |= CS8;
            break;
    }

    switch( nEvent )
    {
    case 'O':
        newtio.c_cflag |= PARENB;
        newtio.c_cflag |= PARODD;
        newtio.c_iflag |= (INPCK | ISTRIP);
        break;
    case 'E':
        newtio.c_iflag |= (INPCK | ISTRIP);
        newtio.c_cflag |= PARENB;
```

```
    newtio.c_cflag &= ~PARODD;
    break;
case 'N':
    newtio.c_cflag &= ~PARENB;
    break;
}

switch( nSpeed )
{
    case 2400:
        cfsetispeed(&newtio, B2400);
        cfsetospeed(&newtio, B2400);
        break;
    case 4800:
        cfsetispeed(&newtio, B4800);
        cfsetospeed(&newtio, B4800);
        break;
    case 9600:
        cfsetispeed(&newtio, B9600);
        cfsetospeed(&newtio, B9600);
        break;
    case 115200:
        cfsetispeed(&newtio, B115200);
        cfsetospeed(&newtio, B115200);
        break;
    case 460800:
        cfsetispeed(&newtio, B460800);
        cfsetospeed(&newtio, B460800);
        break;
    default:
        cfsetispeed(&newtio, B9600);
        cfsetospeed(&newtio, B9600);
        break;
}
if( nStop == 1 )
    newtio.c_cflag &=  ~CSTOPB;
else if ( nStop == 2 )
    newtio.c_cflag |=  CSTOPB;
    newtio.c_cc[VTIME]  = 100;///* 设置超时10 seconds*/
    newtio.c_cc[VMIN] = 0;
    tcflush(fd,TCIFLUSH);
```

```
    if((tcsetattr(fd,TCSANOW,&newtio))!=0)
    {
        perror("com set error");
        return -1;
    }

    return 0;
}
```

这个代码的内容比较简单，主要的部分就是回显。用xshell打开两个串口，接着用arm-linux-gnueabihf-gcc编译好程序之后，通过python http & wget下载到开发板，重新chmod +x之后就可以开始运行了。不出意外的话，就能在/dev/ttyS2的串口下面实现数据的回显功能了。

Open: 68f0752929f66fe221de225f0c647cec_MD5.png



市面上关于串口的设备还是很多的，比如电池、gps、运动控制、二维码结果输出等等，很多设备都是这种232接口的设备，所以小伙伴多多掌握linux下面232如何编程还是大有裨益的。